Lab 08

Disks & Filesystems

CEG 2350 • Leads Aiden Cox & Austin Kellough

Aiden Cox - CEG 2350 Lab Lead
Austin Kellough - CEG 2350 Lab Assistant
Quote of the week:
“Its not a bug, its an undocumented feature”

⚠️ Before You Begin

Read this before touching anything!

Two Sandboxes Required

- Your current sandbox is working — don't break it!
- Create a SECOND AWS sandbox for this lab
- Follow Getting Started with AWS Academy steps 8–13
- Use the same private key; it'll have a new Public IP
- Both instances share the same username: ubuntu

```
ubuntu@ceg2350-sandbox$
sudo hostnamectl set-hostname
disklab-sandbox

ubuntu@ceg2350-sandbox:~$
exit
# reconnect — now shows:
ubuntu@disklab-sandbox:~$
```

Why two sandboxes?

Partition table and filesystem commands are destructive by nature. If you make a mistake on xvda (your main disk), your working instance with your SSH keys and cloned repo is gone. The second sandbox is disposable — make mistakes there safely.

Part 1 — SSH Config File

Managing two sandboxes without losing your mind

Option	Description / Purpose
`Host`	Alias you type: ssh <Host>
`HostName`	Actual IP address or domain
`User`	Login username (e.g. ubuntu)
`Port`	SSH port (default 22)
`IdentityFile`	Path to your private key (~/.ssh/labsuser.pem)

```
# i like to put my original ssh command here commented to grab
# from → ssh -i ~/Keys/ceg2350.pem ubuntu@29.29.144.76

Host
 sandbox1
    HostName     <Public-IP-1>
    User         ubuntu
    IdentityFile ~/your/path/to/keyfile

Host
 disklab
    HostName        <public-IP-2>
    # so on...
```

File Locations

- macOS/Linux: ~/.ssh/config
- Windows: C:\Users\UR-USERNAME\.ssh\config
- Create folder if it doesn't exist: mkdir ~/.ssh
- Test with: ssh <Host_block_name>

Part 2 — What Do We Have?

Exploring xvda — do NOT modify it!

Only query xvda — never modify it in this section

```
$ lsblk /dev/xvda
NAME     MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
xvda     202:0    0   16G  0 disk
├─xvda1  202:1    0   15G  0 part /
├─xvda14 202:14   0    4M  0 part
├─xvda15 202:15   0  106M  0 part /boot/efi
└─xvda16 259:0    0  913M  0 part /boot
xvdb     202:16   0    4G  0 disk  ← no partitions!
```

```
$ sudo parted /dev/xvda print
Partition Table: gpt
Number  Start    End      Size     Type
  1     1049kB   16.1GB   15.0GB   primary
  14    1049kB   5243kB   4194kB   bios_grub
  15    5243kB   116MB    111MB    EFI
  16    116MB    1074MB   958MB    linux-swap
```

Partition Table	# Partitions	Root Filesystem	Mounted At
GPT	4 (xvda1, 14, 15, 16)	/dev/xvda1 → ext4	/ (root)

Part 2 (cont.) — /etc/fstab

The filesystem table — what mounts on boot

```
$ cat /etc/fstab
# <file system>  <mount point>  <type>  <options>  <dump>  <pass>
LABEL=cloudimg-rootfs  /  ext4  discard,errors=remount-ro  0  1
LABEL=BOOT  /boot  ext4  defaults  0  2
LABEL=UEFI  /boot/efi  vfat  umask=0077  0  1
```

<file system>

Device, UUID, or LABEL identifying the partition to mount

<mount point>

Directory where the filesystem is attached (e.g. /)

<type>

Filesystem type: ext4, vfat, swap, auto…

<options>

Mount options. 'defaults' = rw, suid, exec, auto, nouser, async

<dump>

0 = skip backup, 1 = include in dump utility backup

<pass>

fsck order on boot: 0=skip, 1=root first, 2=other partitions

Part 3 — Something New

Partitioning xvdb with gdisk

gdisk Main Menu Options	
p	Print the partition table
o	Create a new empty GPT partition table
n	Add a new partition
i	Show detailed info about a partition
w	Write changes to disk & exit

After gdisk — verify with:

- lsblk /dev/xvdb (see xvdb1 appear)
- sudo parted /dev/xvdb print
- xvdb1 is ~4GB, type: Linux filesystem (8300)

```
$
sudo gdisk /dev/xvdb
GPT fdisk (gdisk) version 1.0.10
Command (?):
o
  → Create new empty GUID partition table? Y
Command (?):
n
  Partition number: 1   (default)
  First sector: [Enter]  Last sector: [Enter]
  Hex code: 8300  (Linux filesystem)
Command (?):
w
  → Changes written to /dev/xvdb
```

Hex code 8300 = Linux Filesystem | Use [Enter] to accept default first/last sectors (full disk)

Part 4 — File It Away

Creating a filesystem on xvdb1 and mounting it

```
# 1. Create ext4 filesystem
$
sudo mkfs.ext4 /dev/xvdb1
mke2fs 1.46.5  Creating filesystem with...

# 2. Verify with blkid
$
sudo blkid /dev/xvdb1
/dev/xvdb1: TYPE="ext4" PARTUUID="..."

# 3. Create mount point & mount
$
sudo mkdir /mnt/expanse
$
sudo mount /dev/xvdb1 /mnt/expanse

# 4. Create files, then unmount
$
sudo touch /mnt/expanse/hello.txt
$
sudo umount /dev/xvdb1
```

Key Concepts

- mkfs.ext4 formats the partition — this DESTROYS existing data
- mount attaches the partition to the directory tree
- You can ONLY interact with files when the partition is mounted
- After umount, /mnt/expanse appears empty — data is still on disk
- blkid shows UUID — useful for fstab entries

Q: When can I interact with files on xvdb1?

A: Only when the filesystem is mounted to a directory. Without mounting, the raw partition is inaccessible via normal file operations.

Part 5 — Take a fstab at This

Auto-mounting xvdb1 on every boot

```
# Step 1: backup fstab FIRST
$
sudo cp /etc/fstab /etc/fstab.bak

# Step 2: get UUID of xvdb1
$
sudo blkid /dev/xvdb1
/dev/xvdb1: UUID="abcd-1234-..." TYPE="ext4"

# Step 3: append to /etc/fstab
$
sudo vim /etc/fstab
UUID=abcd-1234  /mnt/expanse  ext4  defaults  0  2

# Step 4: test (unmount first!)
$
sudo umount /dev/xvdb1
$
sudo mount -a
$
lsblk  # verify xvdb1 shows /mnt/expanse
```

⚠️ Safety Rules

- ALWAYS backup fstab before editing!
- A broken fstab can prevent system boot
- Prefer UUID over /dev/xvdb1 (device names can change)
- If unsure, restore from .bak: sudo cp /etc/fstab.bak /etc/fstab
- mount -a applies fstab without rebooting — test this way!

fstab Entry Breakdown

`UUID=...`	Device (use UUID)
`/mnt/expanse`	Mount point
`ext4`	Filesystem type
`defaults`	Standard options
`0`	Dump (skip)
`2`	fsck pass order

Part 6 — Dead Files May Still Be Read

Data persistence & secure deletion — strings vs shred

Deleting a file removes the directory entry — NOT the data on disk.

```
# create two secret files on xvdb1
$
echo "secret1: my cat is named Whiskers" >> /mnt/expanse/s1.txt
$
echo "secret2: I failed CS1180" >> /mnt/expanse/s2.txt

# run strings on the raw partition
$
sudo strings /dev/xvdb1 | grep secret
secret1: my cat is named Whiskers
secret2: I failed CS1180

# Delete s1 — then check again
$
sudo strings /dev/xvdb1 | grep secret
```

```
# Shred s2 — overwrite bytes on disk
$
sudo YOUR_SHRED_COMMAND /mnt/expanse/s2.txt
shred: /mnt/expanse/s2.txt: pass 1/4
(random)...
shred: /mnt/expanse/s2.txt: pass 2/4
(0x000000)...
shred: /mnt/expanse/s2.txt: removing

# Verify it's gone from disk too
$
sudo COMMAND_TO_VERIFY
```

common shred flags

`-u`	remove file after shredding
`-z`	final pass writes zeros
`-v`	verbose output